

Tema 2 LFA

Precizări

Tema se realizează individual și fără utilizarea modelelor de tip LLM.

Deadline: 10.05.2026, ora 23:59.

Nerespectarea formatului specificat pentru input și output va fi notată cu 0 puncte.

Cerințe

1. Transformare λ -NFA $\rightarrow$ DFA minim

Implementați un program care:

- Transformă un automat finit nedeterminist cu tranziții λ (λ -NFA) într-un automat finit determinist (DFA), utilizând metoda submulțimilor (subset construction).
- Elimină tranzițiile λ prin calculul λ -închiderii (lambda-closure) pentru fiecare stare.
- Minimizează DFA-ul rezultat folosind un algoritm de minimizare.

Fișier Input (fiecare linie este o linie din input):

- Mulțimea stărilor
- Alfabetul de intrare
- Numărul de tranziții, inclusiv tranzițiile λ
- Pe câte o linie, fiecare tranziție, sub forma:

stare_pornire stare_finală simbol

- Starea inițială
- Mulțimea stărilor finale

Fișier Output:

- DFA echivalent, fără tranziții λ
- DFA minim, cu număr minim de stări

2. Transformare λ -NFA $\rightarrow$ Expresie Regulată

Implementați un program care transformă un λ -NFA într-o expresie regulată echivalentă.

Cerințe:

- Automatului i se pot adăuga o stare inițială nouă și o stare finală nouă pentru a simplifica procesul.
- Se vor elimina treptat stările intermediare, actualizând etichetele tranzițiilor cu expresii regulate.

Fișier Input (fiecare linie este o linie din input):

- Mulțimea stărilor
- Alfabetul de intrare
- Numărul de tranziții, inclusiv tranzițiile λ
- Pe câte o linie, fiecare tranziție, sub forma:

stare_pornire stare_finală simbol

- Starea inițială
- Mulțimea stărilor finale

Fișier Output:

- O expresie regulată echivalentă cu limbajul recunoscut de automat

3. Generarea tuturor cuvintelor de lungime fixă folosind o gramatică dată

Implementați un program care generează toate cuvintele de lungime fixă care pot fi obținute folosind o gramatică dată.

Cerințe:

- Programul trebuie să citească definiția unei gramatici și o lungime X .
- Trebuie să genereze toate cuvintele de lungime X care aparțin limbajului generat de gramatică.
- Trebuie tratate producțiile care conțin simboluri terminale și neterminale.
- Trebuie evitate derivările infinite, prin limitarea derivărilor care nu mai pot produce cuvinte de lungime X .
- Cuvintele generate nu trebuie să apară de mai multe ori în output.

Fișier Input (fiecare linie este o linie din input):

- Mulțimea simbolurilor neterminale
- Mulțimea simbolurilor terminale
- Numărul de producții
- Pe câte o linie, fiecare producție, sub forma:

neterminal șir_de_simboluri

unde:

- **neterminal** este simbolul din partea stângă a producției;
- **șir_de_simboluri** poate conține simboluri terminale și neterminale;
- **șir_de_simboluri** poate fi λ , dacă producția generează cuvântul vid.
- Simbolul de start
- Lungimea X a cuvintelor care trebuie generate

Fișier Output:

- Toate cuvintele de lungime X generate de gramatică, câte unul pe linie
- Dacă nu există niciun astfel de cuvânt, se va afișa mesajul **NU EXISTA**

4. Bonus: Transformare Expresie Regulată $\rightarrow$ λ -NFA

Se cere implementarea transformării unei expresii regulate într-un λ -NFA echivalent.

Cerințe:

- Parsarea expresiei regulate utilizând algoritmul Shunting Yard pentru a obține forma postfixată.
- Construirea automatului folosind metoda lui Thompson.
- Tratarea operatorilor: concatenare, reuniune ($|$), steaua Kleene ($*$).

Fișier Input:

- O expresie regulată, ca șir de caractere

Fișier Output:

- Un λ -NFA echivalent: stări, tranziții, stare inițială, stare finală